



## 5. Clase 5

Clase N° 5: Apache Hadoop como framework de software para ejecutar aplicaciones distribuidas y escalables y su aplicación en Big Data. Apache Hadoop e introducción a Apache Hive

### Contenido:

En la clase de hoy trabajaremos los siguientes temas:

?

- Framework Hadoop
- MapReduce
- Ecosistema hadoop
- Comparativa de soluciones para implementar Apache Hadoop
- Apache Hive y su interacción con Hadoop

## 1. Presentación

Hola y bienvenidos al curso de Procesamiento de datos.

En esta sexta semana de clases estudiaremos el entorno de software Hadoop y su utilidad para el Big Data, aprenderemos por qué es necesario contar con este tipo de software y cómo funciona su arquitectura y como MapReduce se utiliza como modelo de programación paralela distribuida. Para irnos metiendo en este framework te invito a ver el siguiente video "Hadoop en 5 minutos"



## 2. Desarrollo y actividades

¿Qué son los datos masivos o Big Data?

Los datos masivos, también conocidos como Big Data, son datos que cumplen entre otras las siguientes condiciones (conocidas como las 3 Vs):

- Tienen gran volumen. Estamos hablando de terabytes o petabytes de información, al menos.
- Tienen gran velocidad. Son datos que varían muy rápidamente, lo que hace que su tiempo de vida sea muy limitado. Esto impone severas restricciones temporales a su almacenamiento y procesamiento, ya que si no se utilizan las técnicas apropiadas, estos datos no se podrán aprovechar convenientemente.

- Tienen una gran variedad. No están limitados a texto, sino que incluyen cualquier tipo de dato, como por ejemplo vídeo, audio, imágenes.

Estas condiciones hacen que los sistemas de almacenamiento y procesamiento de datos tradicionales (como por ejemplo las bases de datos relacionales tradicionales) no sean los más indicados para trabajar con Big Data – esto se verá en mayor detalle más adelante en este documento. Por eso, se han desarrollado tecnologías que permiten el trabajo con datos de estas características, una de las más ampliamente utilizadas es Hadoop.

El Big Data se puede aprovechar en una gran cantidad de campos. Por ejemplo, en temas médicos (expedientes médicos, resultados de pruebas, ...), negocios (compras, ventas, transacciones entre empresas, movimientos de la Bolsa, Business Intelligence ...), redes sociales (mensajes de Twitter, Facebook, ...), o servidores de Internet (logs que recogen los accesos que reciben los servidores, ...).

En esta práctica vamos a introducir la herramienta de Big Data conocida como Hadoop. Profundizaremos en su estructura y en sus componentes principales (el sistema de archivos HDFS y el modelo de programación MapReduce), presentaremos algunos ejemplos de funcionamiento.

## ¿ Qué es Hadoop?

Hadoop es un entorno software para el almacenamiento, procesamiento y análisis de datos masivos, también conocidos como Big Data . Entre sus características más importantes se encuentran las siguientes:

Hadoop es distribuido:

Se ejecuta en un cluster de ordenadores, un conjunto de ordenadores conectados entre ellos mediante una red de interconexión que funcionan de forma coordinada (ver Figura 1).

El cluster permite que el usuario del sistema no tenga que preocuparse de realizar tareas tales como decidir en qué ordenador se ejecutan los trabajos, o de iniciar sesión en la máquina adecuada.

El cluster ofrece una imagen única de sistema (Single System Image, SSI) a sus usuarios, de forma que estos no tienen que ser conscientes de las infraestructuras subyacentes.

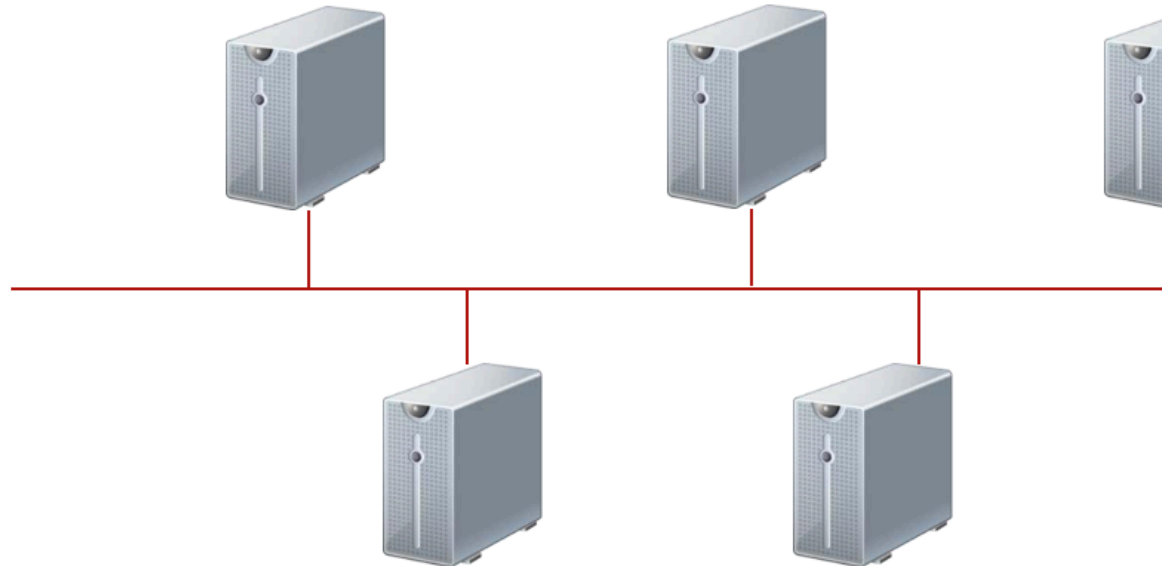


Figura 1. Cluster de ordenadores

Hadoop es escalable:

Añadir nodos incrementa la capacidad de forma proporcional (ver Figura 2). Al contrario que otras tecnologías, que al incrementar el número de nodo implica un sobrecoste, en Hadoop el incremento de los nodos del cluster incrementa directamente la capacidad de procesamiento.

Hadoop es tolerante a fallos:

Los fallos son normales en sistemas distribuidos. En grandes sistemas informáticos, como, por ejemplo, Google, que está formado por cientos de miles de ordenadores, suceden fallos a diario.

El fallo de un nodo no afecta al sistema, que continúa funcionando. El sistema debe ser capaz de detectar el fallo y adaptarse a él con el fin de seguir proporcionando el servicio mientras el fallo se soluciona.

El maestro reasigna las tareas a otro nodo. De esta forma, el servicio sigue siendo proporcionado por las máquinas que siguen en funcionamiento.

No hay pérdida de datos gracias a la replicación. No solamente el servicio se debe seguir proporcionando, sino que la información almacenada debe seguir estando disponible sin pérdidas. Cuando un nodo se recupera, vuelve al sistema automáticamente.

Hadoop es open-source:

Su código fuente está disponible de forma abierta para que quien lo desee lo descargue, modifique

Una gran cantidad de desarrollos software se publican de forma open-source (ver Figura 2), entre los más conocidos se encuentran el sistema operativo para smartphones Android, el navegador de Internet Mozilla Firefox, o el cliente de correo electrónico Mozilla Thunderbird.



Figura 2. Proyectos open-source

## Necesidad de Hadoop

La creciente necesidad de datos hace que los sistemas distribuidos tradicionales no sean eficientes. El principal problema que presentan es el almacenamiento de los datos, ya que se suelen almacenar en una base de datos externa a los nodos de cómputo, tal como muestra la Figura 3.

De esta forma, cada vez que se inicia un procesamiento sobre los datos, los nodos de trabajo deben recuperar la información de la base de datos, lo cual supone un cuello de botella ya que todos los nodos implicados en esos cálculos deben recuperar los datos prácticamente al mismo tiempo y algunos nodos deberán esperar, lo cual retrasa el comienzo de los cálculos y afecta negativamente a la productividad del sistema informático.

En cambio, Hadoop proporciona una nueva gestión de los datos para eliminar ese cuello de botella, como se muestra en la Figura 4. En Hadoop, los datos se almacenan en los mismos ordenadores donde se realizarán los cálculos, de forma que se distribuyen entre ellos en el momento en que se almacenan en el sistema. Por tanto, cuando se inicia un procesamiento, los datos ya se encuentran en los nodos de trabajo, por lo que pueden empezar a trabajar sin demora. Debido a esto, la productividad del sistema informático mejora considerablemente.

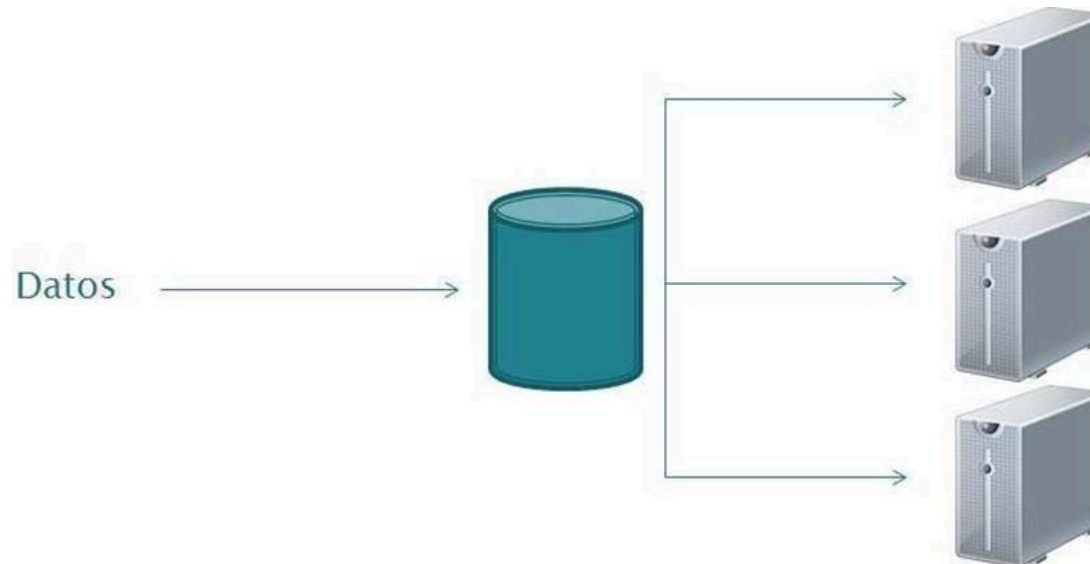


Figura 3. Arquitectura tradicional de un sistema distribuido

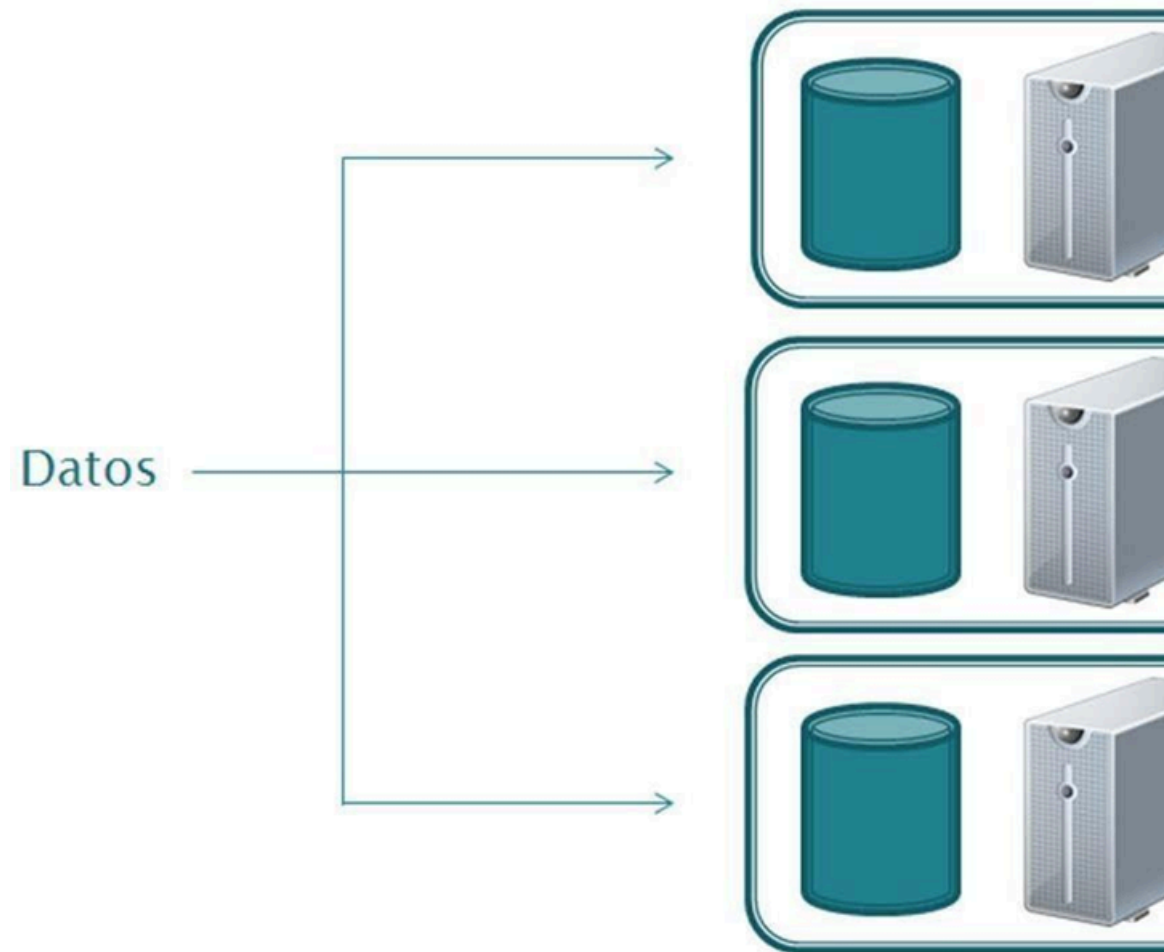


Figura 4. Arquitectura de Hadoop

Cuando los datos se almacenan en Hadoop, estos se dividen en bloques, que es la unidad en la cual se realizan los cálculos (procesos map). Un nodo maestro gestiona los cálculos para asegurar que se realizan correctamente. Este proceso se muestra

gráficamente en la Figura 5.

De esta forma, se persigue que los nodos se comuniquen entre ellos lo menos posible. Como los datos se distribuyen cuando se almacenan, evitamos el cuello de botella que supone leer los datos de la base de datos. Además, los cálculos se llevan a los datos, no al revés, es decir, que los cálculos se ejecutan en los ordenadores que almacenan los datos sobre los que se deben realizar tales cálculos.

Finalmente, Hadoop replica los bloques en varios nodos por razones de prestaciones y de tolerancia a fallos.

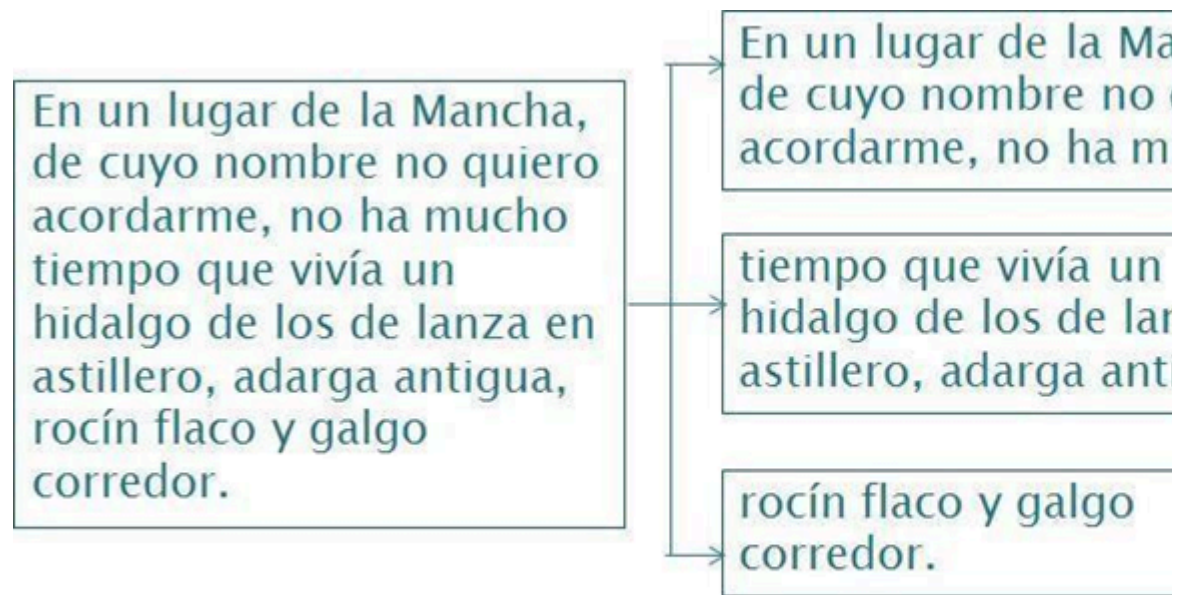


Figura 5. Datos divididos en bloques

## Hadoop y su estructura

Hadoop forma parte de un ecosistema con múltiples componentes, algunos de los cuales se muestran en la Figura 6. Los componentes principales de Hadoop son el sistema de ficheros distribuido de Hadoop (Hadoop Distributed File System, HDFS ) y el entorno de programación MapReduce. A continuación, veremos brevemente las características principales de cada uno.

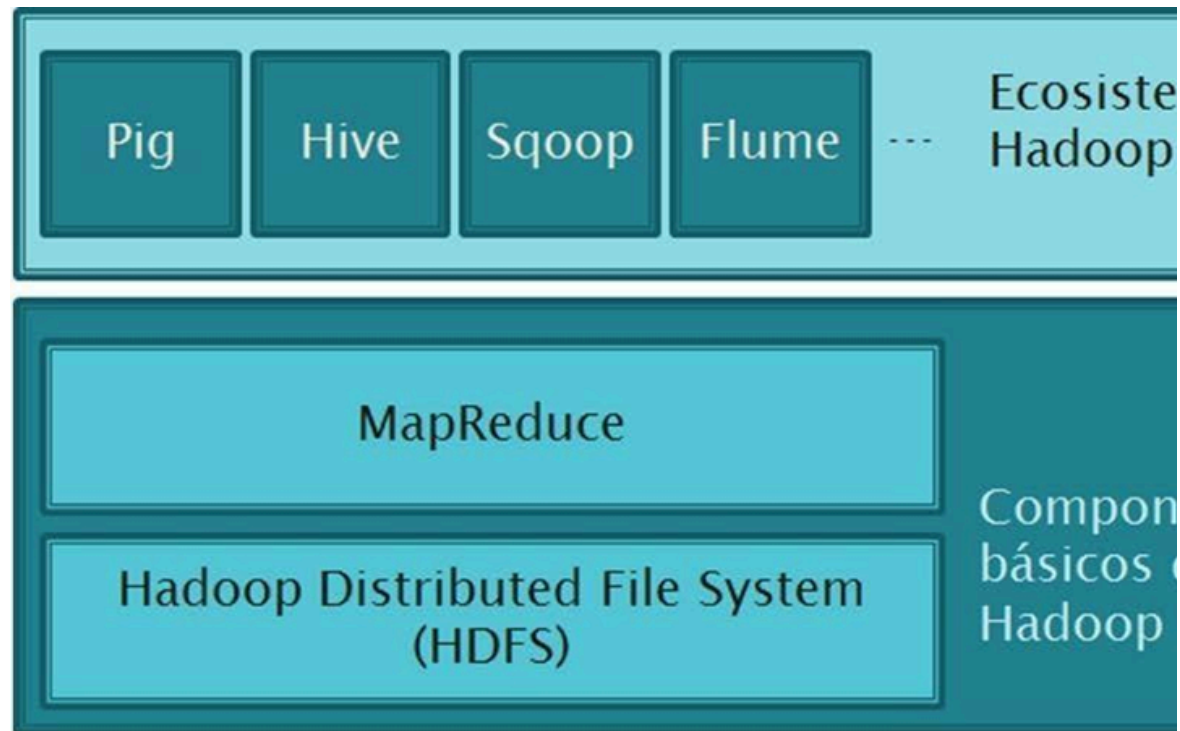


Figura 6. Ecosistema de Hadoop

## Hadoop Distributed File System (HDFS)

HDFS es un sistema de archivos distribuido y tolerante a fallos. Funciona sobre el conjunto de los nodos de un cluster de Hadoop, balanceando la carga de archivos entre las máquinas del cluster, de forma equitativa. Gracias a su naturaleza distribuida, proporciona alta disponibilidad y altas prestaciones que le permiten ser capaz de manejar grandes ficheros.

Para insertar datos en HDFS existen una variedad de formas:

- Copiarlos manualmente utilizando un comando.
- Utilizando la herramienta Flume, que recoge datos de diversas fuentes y los inserta automáticamente.
- Utilizando la herramienta Sqoop, que transfiere datos entre HDFS y bases de datos relacionales.

Ahora vamos a ver con detalle los procesos de escritura y lectura de datos en HDFS.

## Escrituras en HDFS

La forma en que HDFS gestiona las escrituras de archivos se explica a continuación:

1. Primero, el fichero de datos se divide en bloques de tamaño fijo, normalmente 64 o 128 MB. Esto se muestra en la Figura 7.
2. Tras esto, cada bloque se almacena en varios de los nodos del cluster. Esto se muestra en la Figura 8.

De esta forma, al estar cada bloque de datos replicado en varios nodos del cluster, en caso de fallo de alguno de los nodos, no se pierde información, y el sistema puede seguir funcionando correctamente (exceptuando el decremento de la capacidad del sistema consecuencia del fallo).

Para gestionar HDFS, tenemos un nodo especial en el cluster que se llama NameNode. Esta máquina almacena para cada fichero dónde se almacenan los bloques que lo forman.

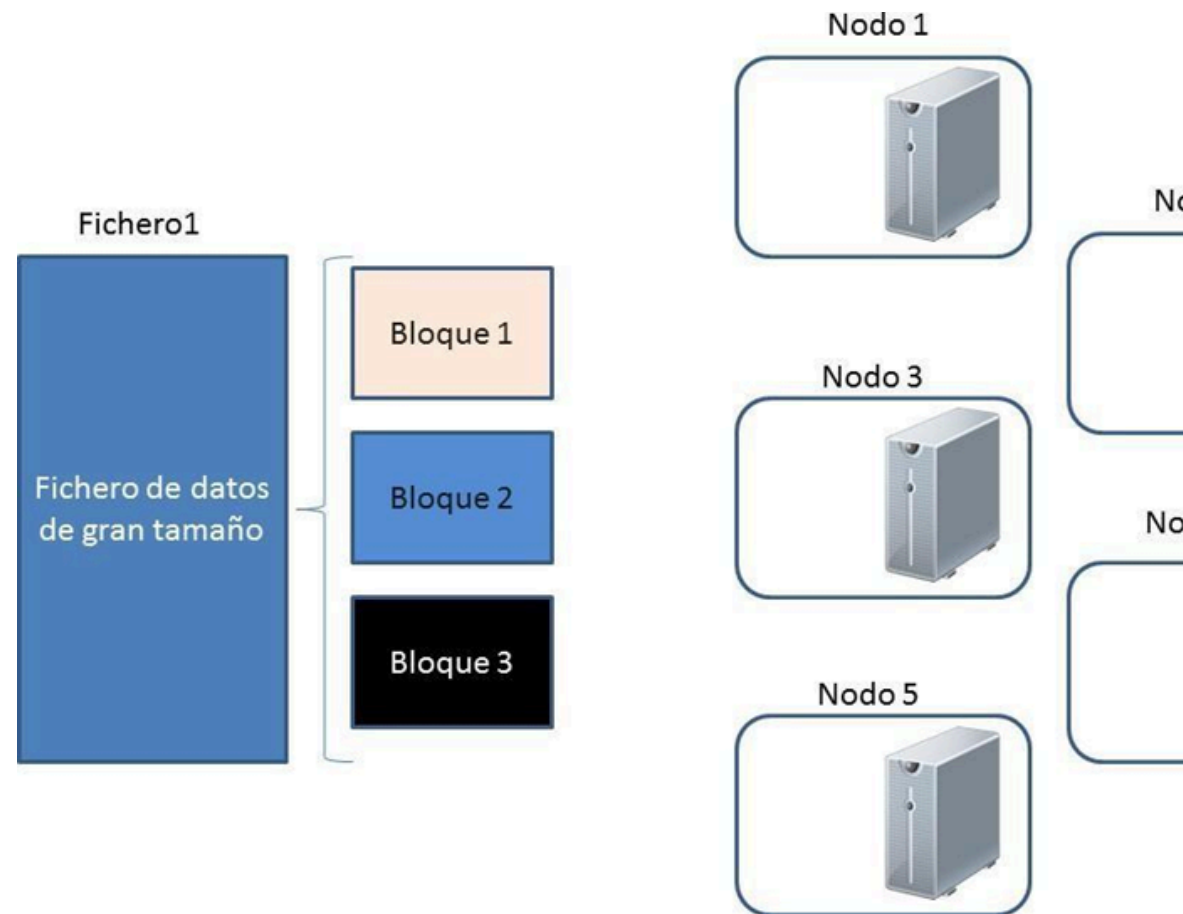


Figura 7. Escrituras en HDFS: Dividir el archivo en bloques

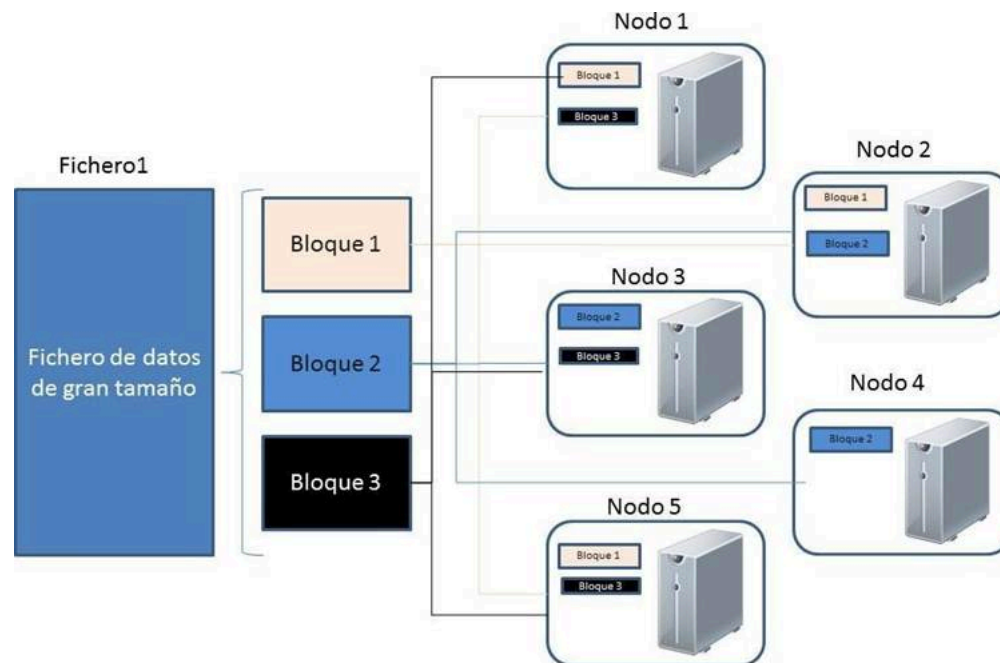


Figura 8. Escritura en HDFS: Almacenar cada bloque en múltiples nodos del cluster

## Lecturas en HDFS

El proceso de lecturas de ficheros en HDFS es como sigue:

1. En primer lugar, el ordenador del cliente realiza la petición de lectura de un archivo al NameNode (esto se muestra en la Figura 9).

2. Entonces, el NameNode chequea en sus registros qué bloques pertenecen a dicho archivo, así como dónde están almacenados tales bloques, y devuelve esta información al cliente (ver Figura 10)
3. Tras esto, el cliente solicita directamente a los nodos correspondientes que le envíen los bloques de dicho archivo (ver Figura 11)

Finalmente, los nodos le envían al cliente los bloques correspondientes directamente, sin pasar por el NameNode.

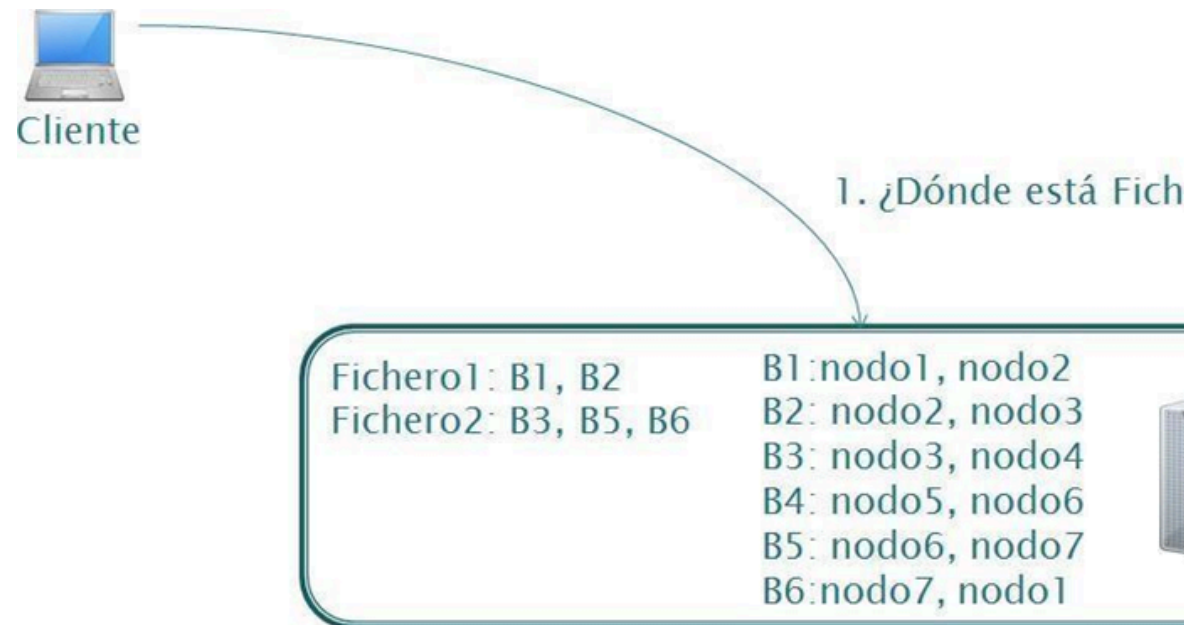


Figura 9. Lectura de HDFS: Petición del cliente

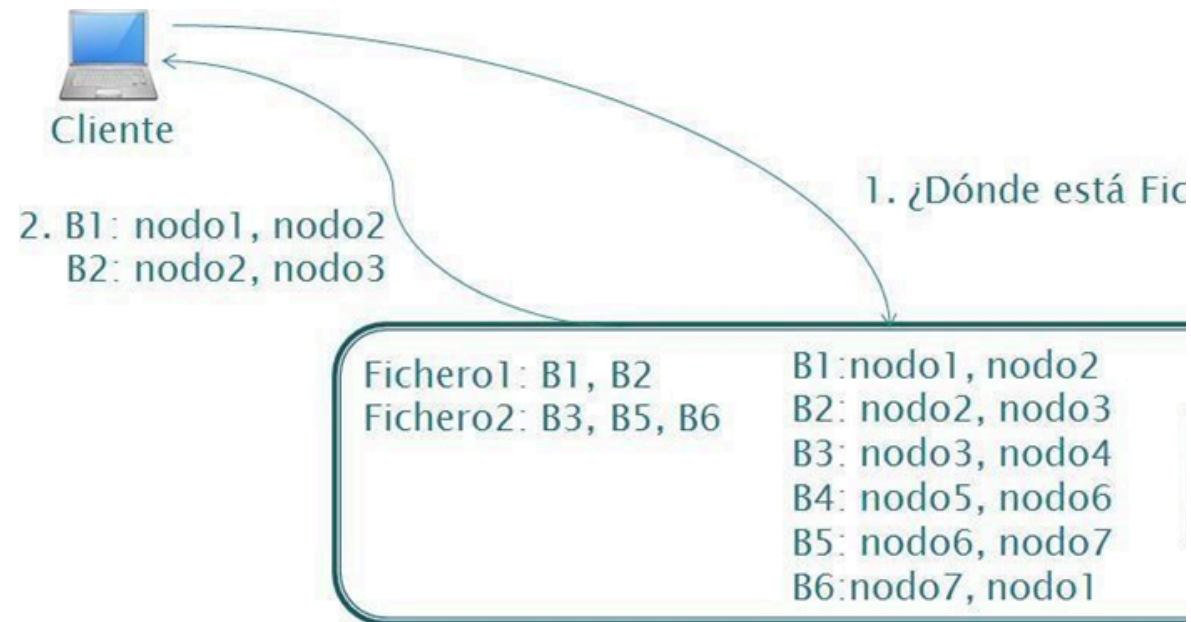


Figura 10. Lectura de HDFS: Respuesta del NameNode

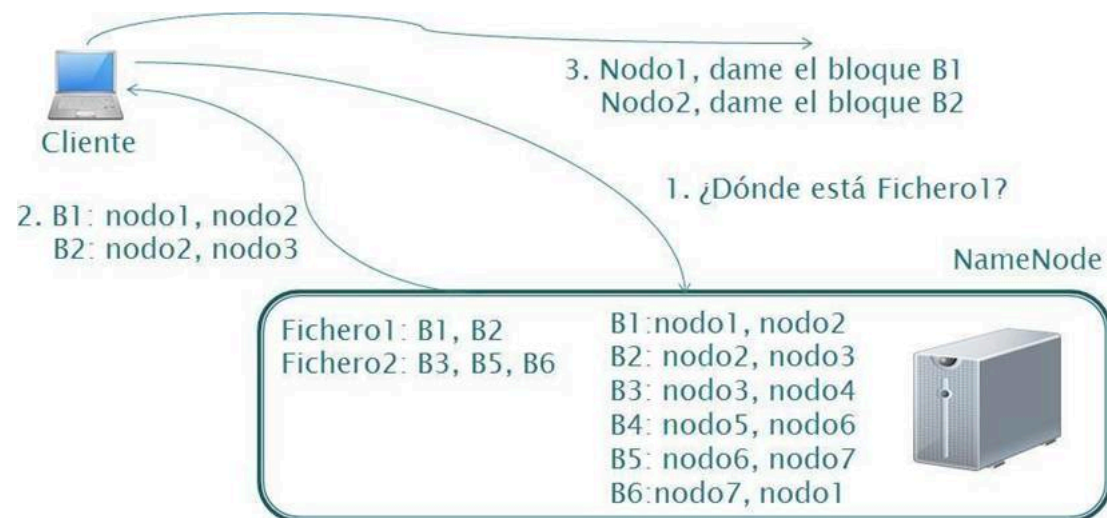


Figura 11. Lectura HDFS: Petición a los nodos

## MapReduce

MapReduce es un modelo de programación paralela distribuida enfocado a grandes conjuntos de datos procesados en un cluster. Automatiza la paralelización de las ejecuciones así como la distribución de las tareas entre los nodos.

MapReduce consta de varias fases:

Map:

- Opera en un único bloque de un fichero HDFS.
- Se ejecuta siempre que sea posible en el nodo que almacena dicho bloque, lo que minimiza el tráfico sobre la red.

Shuffle & sort (barajar y ordenar):

- Ordena y consolida los datos intermedios de todos los maps.
- Sucede cuando todas las tareas map han acabado y antes de que comiencen las tareas reduce.

Reduce:

- Opera sobre los resultados intermedios ordenados y bajados (la salida de las tareas map).
- Produce los resultados finales.

Para entender mejor Mapreduce, vamos a ver un ejemplo típico consistente en un contador de palabras. Partimos de un fichero de texto y deseamos obtener un conteo de las palabras que lo forman, y esto se realiza a través de las funciones Map y Reduce, como se muestra en la Figura 12.

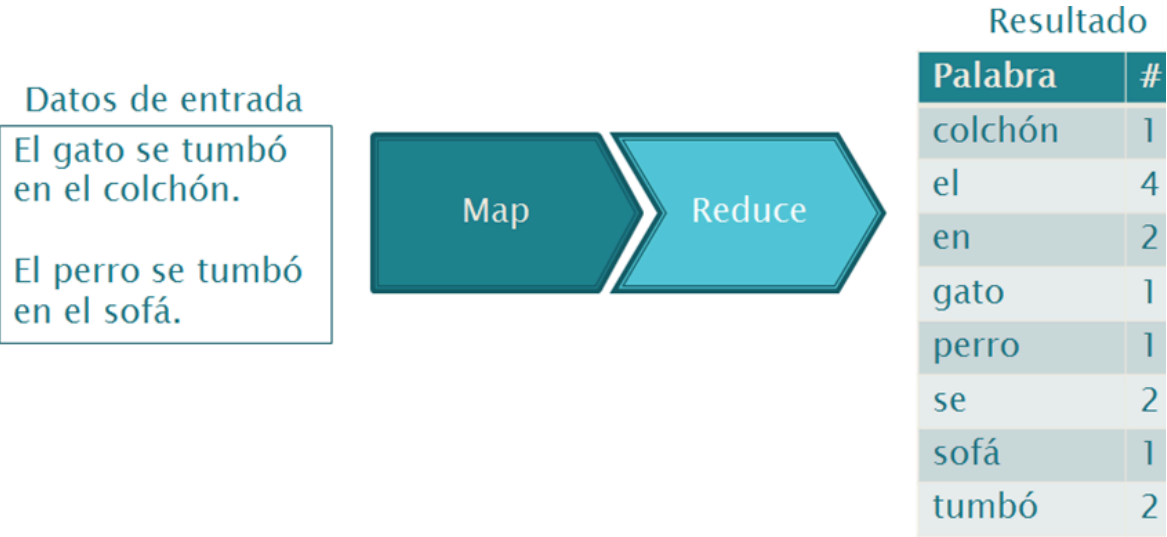


Figura 12. Ejemplo de MapReduce

Los pasos que sigue Hadoop para realizar esta tarea son los que siguen:

1. En primer lugar, se ejecutan procesos map sobre cada bloque que forma dicho fichero. Estos procesos se ejecutan en la medida de lo posible en los ordenadores que almacenan dichos bloques de forma que se minimice la información que se transfiere a través de la red de interconexión. Como resultado de esta fase, se obtiene un listado de pares <clave, valor>, que en este caso tiene como clave cada palabra, y como valor 1. Esto se muestra en la Figura 13.

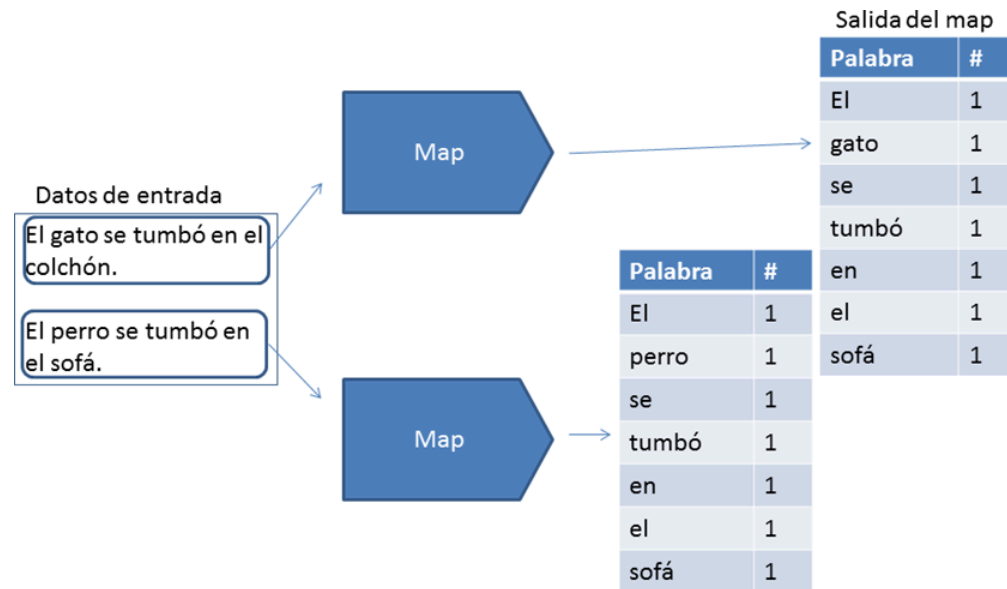


Figura 13. Ejemplo de Mapreduce: Map

2. Tras finalizar los map, la salida de todos los map se combina dentro de la fase de shuffle and sort , de forma que las parejas <clave,valor> que tengan la misma clave se agrupan (es decir, las que tengan la misma palabra)

de la forma en que lo muestra la Figura 14.

3. Finalmente, la fase reduce toma la salida de la fase anterior y agrega los valores que tengan la misma clave, dando como resultado el sumatorio de ocurrencias de cada palabra (ver Figura 15).

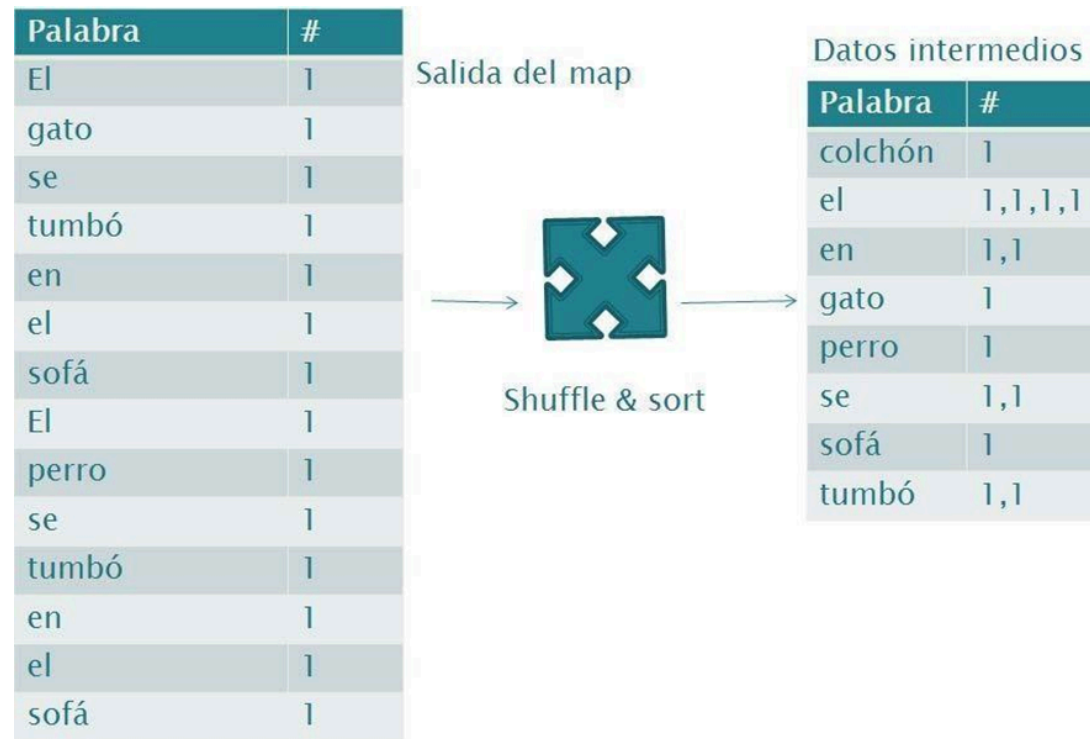


Figura 14. Ejemplo de Mapreduce: Shuffle &amp; sort

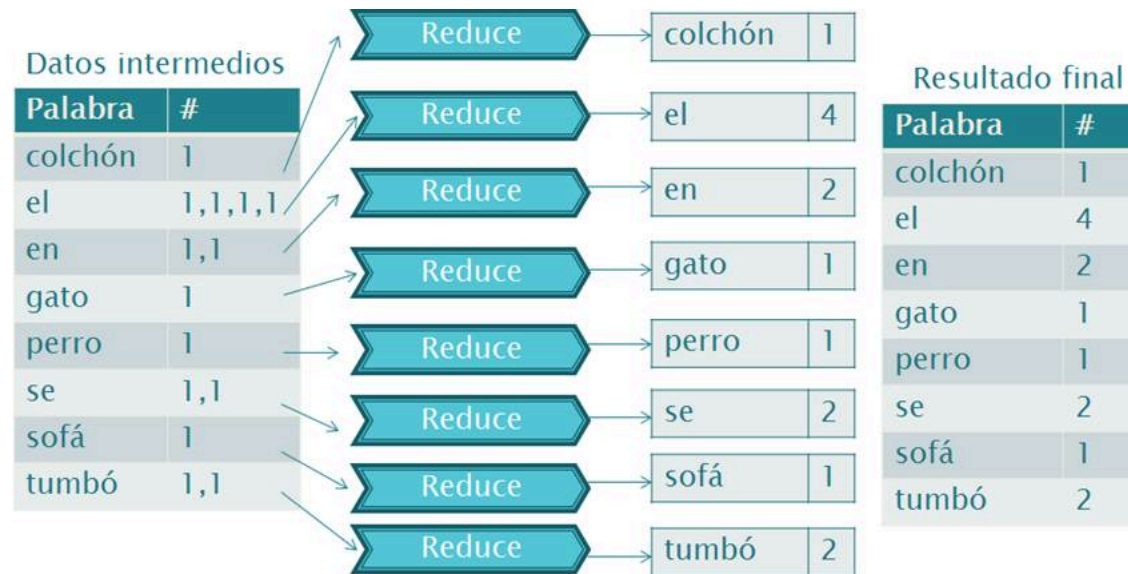


Figura 15. Ejemplo de Mapreduce: Reduce

## Ecosistema de Hadoop

Hadoop es un proyecto vivo con una gran variedad de herramientas que incrementan su funcionalidad y facilidad de uso.

- Hive: Interacción con Hadoop utilizando un lenguaje de consultas similar a SQL.
- Pig: Interacción con Hadoop utilizando un lenguaje de script.
- Sqoop: Comunicación de Hadoop con bases de datos relacionales.
- Flume: Herramienta para mover grandes cantidades de datos.
- HUE: Interfaz gráfico para Hadoop.

# Comparativa de Soluciones Hadoop

Los cinco gigantes del Big Data, Amazon, Apple, Google, Microsoft y Facebook disponen de miles e incluso millones de ordenadores para el tratamiento de sus datos. Estas empresas tienen recursos y herramientas desarrolladas por ellos mismos para la compleja tarea de gestión de sus clústeres.

Comprender la complejidad de la infraestructura de un clúster de Big Data, y la complicación del tratamiento de los datos y su análisis para obtener rendimiento monetario de estos procesos es importante para poder entender mejor los modelos de negocio existentes en el mercado.

Existen dos modelos de negocio. El primer modelo sería las empresas que ofrecen servicios de Hadoop integrales que engloban todos los ámbitos, como son Pivotal, MapR, Microsoft Azure, Amazon AWS, etc. El segundo modelo se especializa en integrar todas las herramientas de Hadoop proporcionando un sandbox que facilite su instalación y manejo, como son, Cloudera Inc., MapR Technologies Inc., IBM InfoSphere Insights, HD Microsoft Insight.

Actividad 1: Investigar sobre los modelos integrales ofrecidos en el mercado: Pivotal Software Inc., MapR, DELL EMC, Amazon AWS, IBM Microsoft Azure, Google Cloud DataProc.

Actividad 2: Investigar sobre los modelos sandbox como Cloudera y HortonWorks. ¿En qué se diferencian de los modelos integrales descritos?

## Teorema CAP

El Teorema CAP (Consistency, Availability, Partition Tolerance) es fundamental en sistemas distribuidos, como los que se usan en Big Data (Hadoop, Cassandra, Kafka, MongoDB, etc.). Establece que en presencia de una partición de red (fallo en la comunicación entre nodos), un sistema distribuido solo puede garantizar dos de estas tres propiedades:

### 1. Consistencia (C)

Todos los nodos ven los mismos datos al mismo tiempo.

Ejemplo: en una base SQL tradicional, si actualizas un valor, cualquier consulta posterior debe devolver el nuevo valor.

### 2. Disponibilidad (A)

El sistema siempre responde a las peticiones (aunque devuelva datos antiguos o incompletos).

Ejemplo: Cassandra o DynamoDB priorizan que siempre obtengas respuesta.

### 3. Tolerancia a particiones (P)

El sistema sigue funcionando aunque haya fallos de red o nodos incomunicados.

Ejemplo: sistemas distribuidos en clusters geográficamente dispersos.

En Big Data, P (tolerancia a particiones) es indispensable (los clusters suelen estar distribuidos).

Entonces, hay que elegir entre:

- CP: Consistencia + Tolerancia a particiones (ej: HBase, Zookeeper).
- AP: Disponibilidad + Tolerancia a particiones (ej: Cassandra, CouchDB).

## Principio SCV (Speed – Consistency – Volume)

Este principio se usa como criterio de calidad y diseño de soluciones Big Data. Se parece a los "3V" de Gartner (Volumen, Velocidad, Variedad), pero enfocado en otro ángulo:

### 1. Speed (Velocidad) ⚡

- La capacidad de procesar y analizar datos en tiempo real o casi real.
- Ejemplo: Kafka o Spark Streaming para procesar flujos de datos de sensores, transacciones o redes sociales.

### 2. Consistency (Consistencia) ✅

- Garantizar que los datos procesados y almacenados mantengan integridad y coherencia.

- En Big Data suele implicar consistencia eventual, pero la idea es que los análisis no se basen en datos corruptos o contradictorios.
- Ejemplo: en HDFS los bloques se replican (factor 3) para asegurar que los datos no se pierdan y sean consistentes en los nodos.

### 3. Volume (Volumen) 📦

- La capacidad de manejar grandes cantidades de datos (terabytes, petabytes, exabytes).
- Implica almacenamiento distribuido y escalabilidad horizontal (más nodos en el clúster en lugar de máquinas más potentes).
- Ejemplo: Hadoop Distributed File System (HDFS) o Amazon S3 para guardar billones de registros.

No siempre se pueden garantizar los 3 (Speed, Consistency y Volume) al mismo tiempo de forma perfecta en Big Data.

Razón principal

Los sistemas distribuidos y de Big Data siempre trabajan bajo compromisos:

- Si el volumen es muy alto (petabytes), procesar con velocidad puede exigir sacrificar consistencia estricta (se opta por consistencia eventual, se usan muestreos para realizar los análisis).
- Si queremos consistencia fuerte, el sistema se vuelve más lento (pierde speed).
- Si buscamos máxima velocidad y consistencia inmediata, puede que el volumen procesado en paralelo deba ser limitado.

Esto se relaciona con el Teorema CAP: en un entorno distribuido, siempre hay que elegir prioridades.

## Ejemplos concretos

- Redes sociales (Twitter, Facebook, Instagram):  
Priorizan Velocidad + Volumen, pero aceptan que los likes o comentarios tarden segundos en sincronizarse (consistencia eventual).
- Sistemas financieros/bancarios:  
Priorizan Consistencia + Volumen (es crítico que no haya errores de saldos), aunque sean menos veloces.
- Sistemas de recomendación en streaming (Netflix, Spotify):  
Priorizan Velocidad + Volumen para sugerir en tiempo real, aunque la consistencia no sea perfecta.

## Apache Hive

### Logo de Apache Hive



Dentro del ecosistema Hadoop encontramos otros frameworks como es Hive.

Apache Hive (<https://hive.apache.org/>) es una tecnología distribuida diseñada y construida sobre un clúster de Hadoop. Permite leer, escribir y gestionar grandes datasets (con escala de petabytes) que residen en HDFS haciendo uso de un lenguaje dialecto de SQL, conocido como HiveSQL, lo que simplifica mucho el desarrollo y la gestión de Hadoop.

La tecnología Hadoop es altamente escalable, sin embargo, tiene dos problemas principales:

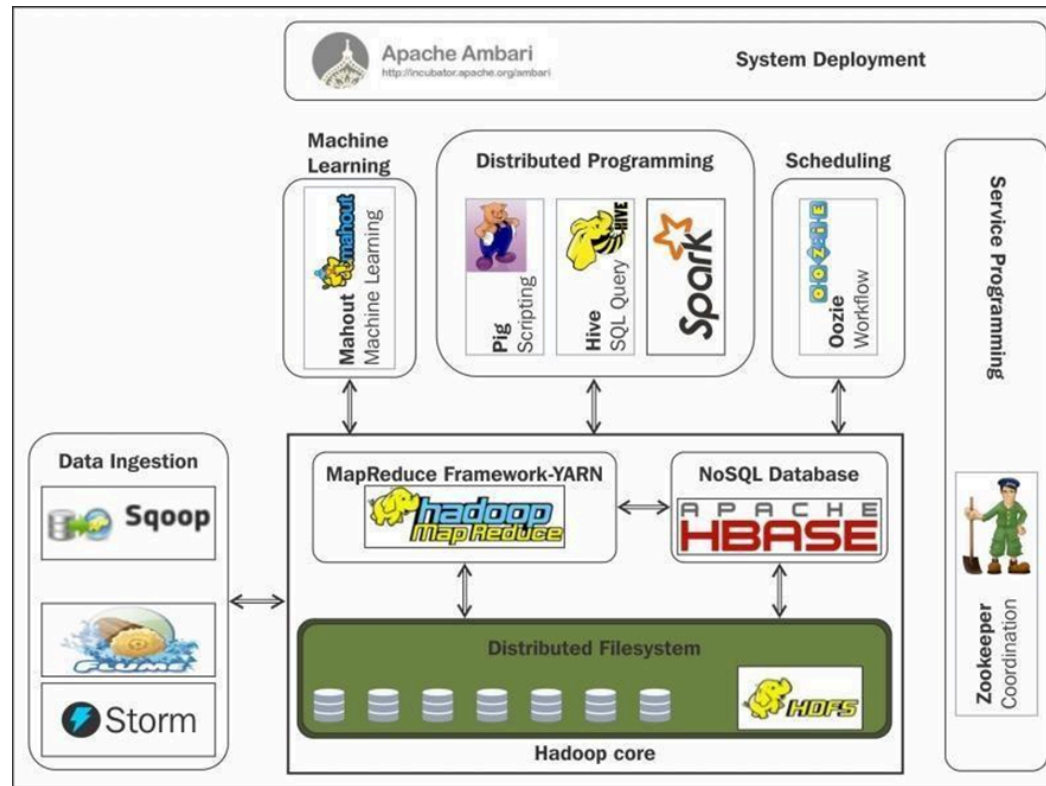
Dificultad de uso: La API de Java es complicada de usar y se necesita experiencia específica para tratar con diferentes formatos de ficheros y sistemas de almacenamiento.

Orientado a operaciones Batch: No soporta operaciones de acceso aleatorio y no está optimizado para gestionar ficheros pequeños.

### Hive y Hadoop

Si volvemos a ver como funciona Hive dentro del ecosistema de Hadoop, Hive es una fachada construida sobre Hadoop que permite acceder a los datos almacenados en HDFS de forma muy sencilla sin necesidad de conocer Java, Map Reduce u otras tecnologías.

Aunque en principio estaba diseñado para el procesamiento batch, ahora se integra con frameworks en streaming como Tez y Spark.



### 3. Instancia de Autoevaluación: Entrega de presentación grabada

En esta instancia de evaluación deberás hacer entrega de la grabación sobre Bases de datos NOSQL según las pautas definidas. Deberá compartirse el video en formato mp4 y disponible en un repositorio público (estilo drive, one drive, etc.)

Es importante tener en cuenta que para aprobar esta instancia deberás:

- Entregar en tiempo y forma

- Realizar un 60% de la actividad correctamente.
- Cumplir con las consignas
- Cumplir con el formato requerido.

## 4. Cierre:

Como cierre de esta primera clase te invito a que reflexiones sobre la utilidad y necesidad de contar con un entorno como Hadoop para el Big Data. También te invito a buscar más herramientas de implementación de este entorno de software que no hemos presentado en esta clase y compartir información y opiniones sobre ellas en el foro de intercambio.

Te esperamos en los próximos encuentros. Recordar que es importante participar de los foros, de los encuentros sincrónicos y en lo posible de las tutorías presenciales.

## 5. Bibliografía Obligatoria:

Du, D. (2018). Apache Hive Essentials: Essential Techniques to Help You Process, and Get Unique Insights From, Big Data, 2nd Edition. Packt Publishing.

Medranom A. Apuntes del curso Inteligencia Artificial y Big Data, I.E.S. Severo Ochoa  
<https://aitor-medrano.github.io/iabd2223/index.html>

Página oficial de Hive <https://hive.apache.org/>

White, T. (2009) Hadoop: The Definitive Guide. O'Reilly  
Media Gravenstein Highway North, Sebastopol.

<https://hadoop.apache.org/>

<http://hive.apache.org/> <http://apache.org/cassandra/>

<http://www.hispabigdata.es/2013/09/como-instalar-apache-cassandra.html>